researchpulse · the six agents

What each agent does, what it hands to the next one, and which parts of Problem Statement 5 it answers.

Problem DejaVu Hackathon · Advanced Agentic AI · PS 5 ResearchPulse **Author** N Sreyesh Kumar

Code github.com/shreynachaegari-glitch/limitless **Date** 14 October 2026

1. The problem statement in one paragraph

PS 5 asks for an agentic system that tracks selected topics and sources, finds relevant new information, compares it with what was found before, and decides whether a development deserves attention, explaining **what changed, why it matters and how confident it is**. It lists nine advanced challenges (topic-aware monitoring, duplicate detection, change detection, historical comparison, trend analysis, significance ranking, source reliability, alert control, traceable summaries) and six evaluation scenarios.

ResearchPulse answers it with six agents that work as one pipeline. Each agent owns one part of the problem, and every monitoring run passes through all six in order:

```
Scout ──► Resolver ──► Analyst ──► Verifier ──► Sentinel ──► Trend
find new    is it relevant,  what changed     can we prove it,   should we       is something
items       and is it new    vs history?      how reliable?      alert?          building up?
```

Rules decide, the model explains. Classification, scores, priority and alerts are computed by deterministic, tested rules, so the six scenarios give the same result every time and a source page can never change a priority. NVIDIA Nemotron is used where language understanding helps: reading claims out of real abstracts, writing the explanation of each change (with at most three look-up tools), and judging borderline duplicates. Everything the model produces passes a code check before it is used.

2. The agents

01 Scout: finds what is new rules + network

Job	Collects new items for a topic from its sources and stores each one exactly once, with every version kept.
Sources	arXiv (preprints), Crossref (peer-reviewed journals and proceedings from Springer Nature, IEEE, ACM, Elsevier), OpenAlex, optional RSS feeds, and the scripted scenario replay.
How	Incremental cursors so only new material is fetched; timeouts, retries with back-off, respects `Retry-After` and arXiv's 3-second rule; responses capped at 5 MB; feed URLs must point to public addresses. Each item gets an identity (DOI, arXiv id or cleaned URL) and a content hash: same hash means "already known", a new hash on a known item is stored as a **new version**, never as a second paper. Instruction-like sentences in a page ("ignore previous instructions...") are detected and quarantined here.
Hands on	New documents and new versions, with source role (primary / secondary / unverified) and per-source health.
You see	Console: "*arXiv API: 23 items, 23 new or changed*". Sources tab: ok / failing / never run, so "no news" is never confused with "fetch failed".
Code	`sources.py`, `ingest.py`, `netsafe.py`

> **PS link:** continuous monitoring of selected sources; **source reliability** (role recorded at the source); **change detection** begins here (new version vs new paper); the basis of **traceable summaries** (every item keeps its original link and text).

02 Resolver: is it relevant, and is it really new? rules + embeddings

model on borderline cases

Job	Drops off-topic items and groups reports of the same development, without merging independent studies.
Relevance	Semantic similarity between the topic profile and the item (BAAI/bge-small embeddings, calibrated), plus concept and synonym matches, minus exclusions. A news report that cites a relevant paper inherits its relevance. Reasons are stored ("matches exclusion 'image generation'").
Grouping	Four tiers: identity → content hash → candidates (citation links, similarity, title overlap, identical claims) → decision. Guards veto a merge when the items measure different models or are two primary papers by different authors. Only the ambiguous band goes to Nemotron, and its verdict must be a strict yes/no; otherwise the items stay separate.
Hands on	Relevant items attached to a development cluster, with the reason; how many reports and how many independent original sources ("3 reports, based on 1 original source").
You see	Console: "*grouped 'NovaQuant shrinks Nova-7B...' with an existing development (cites the same original source)*", "*closest candidate kept separate: different models (lumen-8b vs nova-7b)*".
Code	`resolve.py`, `embed.py`, `narrate.adjudicate_factory`

> **PS link: topic-aware monitoring** (beyond exact keywords); **duplicate detection**; alert control (repeats are recognised before they can alert). Scenario **1 · several articles report the same development**.

03 Analyst: what changed compared with history? rules classify

model reads claims and explains

Job	Compares each item with what the topic already knows and says what kind of change it is.
Claims	Pulls measurable claims (model · metric · value · unit · benchmark · method / batch size / context length), each with the exact span of text it came from. Patterns handle well-formed sentences; for real abstracts Nemotron proposes claims and code keeps one only if its quote is copied word for word and contains the number (latest live run: 26 kept, 28 rejected, all kept quotes exact).
Comparison	Same work, new version → compares old and new values (revision or cosmetic edit). Different works → numbers are compared only if model, metric, unit, benchmark and conditions match; otherwise "not directly comparable" with the reason (for example "different batch size").
Seven kinds	new finding · duplicate report · minor update · material revision · contradiction · related development · unverified claim.
Explains	A bounded Nemotron agent writes the headline, "what changed" and "why it matters" from an evidence packet, with up to three read-only tools (`get_cluster_reports`, `get_history`, `verify_excerpt`), a deadline and one corrective retry. Any number in its text that is not in the evidence gets the text rejected and a template used instead.
You see	Results cards: "*Accuracy loss: earlier report 0.8 points vs new study 6.1*", "*v2 said 2.4x → v3 says 1.9x (−20.8 %)*"; the side-by-side Investigation view.
Code	`analyze.py`, `claims.py`, `llm_claims.py`, `narrate.py`

> **PS link: change detection**; **historical comparison** (claims and values diffed against history); "explain what changed rather than restate the article". Scenarios **2 · contradiction**, **3 · minor update**, **6 · a source changes a reported value**.

04 Verifier: can we prove it, and how reliable is it? rules

Job	Checks the evidence behind each assessment and decides how much weight it deserves.
Evidence check	Every quoted excerpt must exist verbatim in the stored text of the version it cites; a failed check sends the item to review instead of letting it alert.
Reliability	Primary vs secondary vs unverified source; peer-reviewed venue (+8); a report that links its original (+6); independent original sources counted, not articles; promotional language and missing conditions flagged; a page that carried injected instructions loses weight (−25).
Scores	Six separate scores, each with its reason: relevance, novelty, materiality, impact, evidence, urgency. Confidence (high / medium / low) is an explained rule, not a fake percentage. Priority: HIGH, REVIEW, MEDIUM, LOW, NONE.
You see	Score bars with reasons; "high confidence" ticks; ⚑ flags such as "*Weakly supported: no primary source, no measured result*" or "*Prompt injection quarantined*".
Code	`verify.py`, `analyze.score`, `analyze.decide_priority`, `resolve.independence`

> **PS link: source reliability** (primary, secondary, unverified); **significance ranking**; **traceable summaries**; "distinguish reported facts from inferred trends". Scenario **5 · a misleading or weakly supported claim labelled cautiously**.

05 Sentinel: does this deserve an alert? rules

Job Turns assessments into alerts only when they are worth interrupting someone for.

How The first run on a topic is treated as history (no alerts). Duplicates and cosmetic edits never alert; a repeat report bumps the existing alert's "suppressed" count instead. Each topic has an alert level (the scenario topic alerts only on HIGH and REVIEW). Alerts carry an idempotency key enforced by the database, so a rerun or retried job can never send the same alert twice.

You see Console: "*ALERT HIGH: …*", "*3 alerts sent, 8 held back*"; Alerts tab with the unread badge.

Code `pipeline.py` (Sentinel phase), `analyze.should_alert`, `analyze.alert_key`

> **PS link: alert control** ("reduce repetitive or low-value notifications"); significant-change alerts. Scenario **3 · a minor update that should not trigger a high-priority alert**, and the rerun that creates nothing new.

06 Trend: is something building up? rules

Job Finds recurring themes and emerging developments over time, and says when it cannot tell.

How For each topic concept, counts **distinct developments** (clusters, not articles) per 14-day window against the previous 42 days. Developments backed only by unverified sources are not counted. Status: emerging, established, tentative, steady, or **too early to call** when history or data is too thin. Contradicting developments are listed with the trend.

You see Trends tab: "*speculative decoding: emerging — 4 distinct developments in the last 14 days vs 1 in the prior 42*", "*5 articles → 3 developments*", "*too early to call*".

Code `trends.py`

> **PS link: trend analysis**; "evidence-backed trend summaries"; "an indication when there is insufficient evidence to establish a meaningful trend". Scenario **4 · several related updates that collectively reveal a trend**.

3. Requirement → agent map

PS 5 advanced challenge	Scout	Resolver	Analyst	Verifier	Sentinel	Trend
Topic-aware monitoring	○	●				
Duplicate detection	○	●			○	○
Change detection	○		●			
Historical comparison			●	○		
Trend analysis		○				●
Significance ranking			○	●	○	
Source reliability	○	○		●		○
Alert control		○			●	
Traceable summaries	○		○	●		

● owns the requirement · ○ contributes to it

4. Evaluation scenario → agent

PS 5 scenario	Agent that decides it	What the system shows
Several articles report the same development	Resolver, then Sentinel	Two news posts grouped with the NovaQuant paper: "3 reports, based on 1 original source"; no repeat alert
A new report contradicts an earlier one	Analyst	HIGH contradiction: accuracy loss 0.8 vs 6.1 points, same model, method and benchmark; both sources side by side
A minor update that should not alert	Analyst, then Sentinel	Version 2 fixes typos: minor update, priority NONE, no alert
Related updates reveal a trend	Trend	Speculative decoding emerging (4 vs 1); sparse attention "too early to call"
A weakly supported claim	Verifier	"10x faster on any model, guaranteed": unverified claim, REVIEW, low confidence, gaps listed
A source changes a reported value	Analyst, then Sentinel	v2 said 2.4x → v3 says 1.9x (−20.8 %): HIGH revision; the two news posts that repeated 2.4x are flagged as out of date

5. Expected outcomes → where they appear

PS 5 expected outcome	Produced by	Where you see it
An organised feed of topic-relevant updates	Resolver + Verifier	Feed, and the Run results page grouped by "needs your attention / worth a look / for the record"
Summaries that explain what is new	Analyst	One-line change ("v2 said 2.4x → v3 says 1.9x") and "What this means" on every card
Duplicates grouped together	Resolver	"3 reports, based on 1 original source"
Comparison with historical information	Analyst	Investigation view: old vs new, delta, comparability
Evidence-backed trends and change alerts	Trend, Sentinel	Trends tab, Alerts tab
Source references; facts vs inferred trends	Verifier, Trend	Verified excerpts with links; trend limits ("co-occurrence only; no causal claim")
Configurable relevance, fewer repetitive alerts	Resolver, Sentinel	Topics: concepts, exclusions, relevance and alert thresholds
Indication of insufficient evidence for a trend	Trend	"Too early to call"

6. How the agents work together

- **One run, six stages.** A run is a job in PostgreSQL; two worker processes claim jobs safely (`FOR UPDATE SKIP LOCKED`), so a run started from the interface never waits behind the hourly live run. Each stage commits before the next, so a crash loses nothing: a retried job finishes the work.
- **Shared memory.** The agents read and write one store: documents and versions, claims with their exact spans, development clusters, assessments with their evidence, trends and alerts. That shared history is what lets the Analyst compare and the Trend agent count.

- **Visible hand-offs.** Each stage writes a trace line as it works. The interface turns the helix to each agent in order while the console shows its lines, then opens the Run results page for that run.
- **Where the model is, and is not.** Nemotron (`nvidia/nemotron-3.5-lightning-30b-a3b` on NVIDIA NIM) is used inside the Analyst (claims from real abstracts; explanations with up to three tools) and the Resolver (borderline duplicates). Scout, Verifier, Sentinel and Trend are deterministic. If the model is down or rate-limited, the pipeline still completes with template text.
- **Tested.** 33 automated tests in `backend/tests`, run against a real PostgreSQL database, cover the six scenarios, the duplicate and comparability guards, the safety checks and the model guards (verbatim claims, grounded numbers, the tool loop).